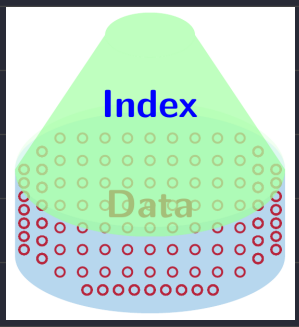
CH7 Maps and Dictionaries

An Index



Index is a **mapping function** or **data structure** that speeds data retrieval from a data set, and it is **one-to-one**

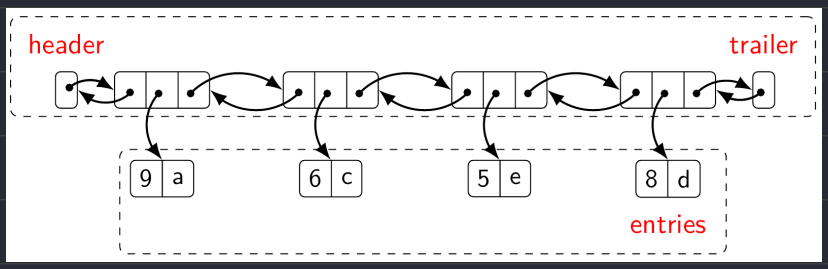
Examples: hash tables, binary search, trees, B-trees

Maps

Definition

A collection of key-value entries (k, v)

List-Based Map



ADT methods

Searching (get(k))	O(n)	最糟情況找不到 key
Inserting (put(k, v))	O(n)	需要檢查 key 是否已經存在
Deleting (remove(k))	O(n)	最糟情況找不到 key

Others (size(), isEmpty())

→ The **unsorted list implementation** is effective only for maps of **small size**

Dictionary

Definition

Like a map, but it is **multiple-to-one**

Implementation

Unordered dictionary : unsorted lists, hash tables

Ordered dictionary : ordered search tables

A List-Based Dictionary

Usage

物件少或多為插入操作適用，e.g. log file, audit trail (數據軌跡)

ADT methods

O(n) find(k) findAll(k) remove(e)

O(1) insert(k, o)

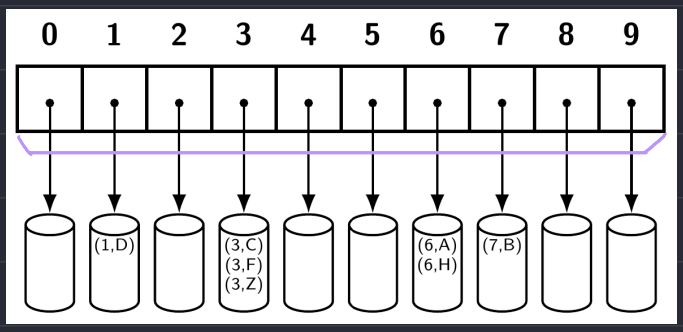
Hash Tables

Definition

The most efficient ways to implement a map

Consist of Bucket array + Hash function

Bucket Arrays



Pros

If keys are unique in the range [0, N - 1], then each bucket holds at most one entry and all the operations can be done in O(1) time.

Cons

Wasting space: N ≫ n, n is the number of entries

Hard to have keys only in the range of [0, N - 1]

Hash Functions

Definition

- Maps each key of a given type to an integer in a fixed interval

  (k, v) at index i = h(k). hash function hash value

- Usually the key (hash code) is from

  1. Memory address, 2. Integer cast, 3. Component sum

Collision

Definition

Two keys thrown into a hash function comes out the same hash value

Compression function

Definition

Solve the collision: [object] → [hash function] → [Compression function]

Polynomial Hash Codes

1. Partition the key object into a sequence of components of fixed length

   e.g. 8-bit a chunk, then get $\{a_0, a_1, a_2, ..., a_n - 1\}$

2. Evaluate the polynomial

   $p(z) = a_0 + a_1 z + a_2 z^2 + a_3 z^3 + ... + a_{n-1} z^{n-1}$

   By Horner's rule, p(z) can be evaluate in O(n)

   $p_0(z) = a_{n-1}$

   $p_i(z) = a_{n-i-1} + z p_{i-1}(z)$ → 遞迴 n 次

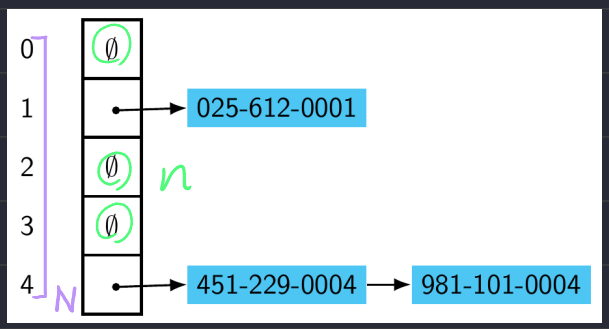
Solving the Collisions

1. Separate Chaining

   - 遇到重複的 key 就往後放
   - 實作相對簡單
   - 需要額外的記憶體儲存往後放的位置

   Load Factor $\lambda = \frac{n}{N}$

   最好 < 1，這樣最高效率



2. Open Addressing Scheme

   ### Definition

   - If space is at a premium (有限), this is the condition to use
   - The load factor is at most 1
   - The entries are stored directly in the cells

   ### Methods

   linear probing, quadratic probing, double hashing

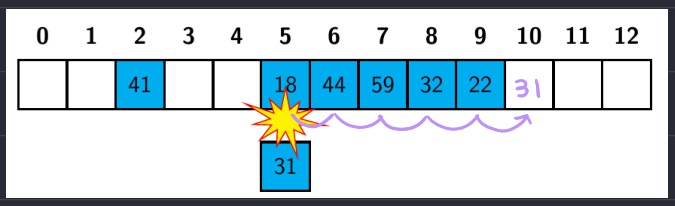
Linear Probing

Search [get(k)]

Start with h(k) 然後往後找直到 碰到空格子(找不到) 或 碰到對應的k的資料 (回傳)

Inserting [put(k, o)]

發生碰撞時就往後移動，直到可以放入為止



h(18) = 5

h(31) = 5

Deleting [remove(k)]

Start with h(k), if data with key k found, then remove it

Double Hashing

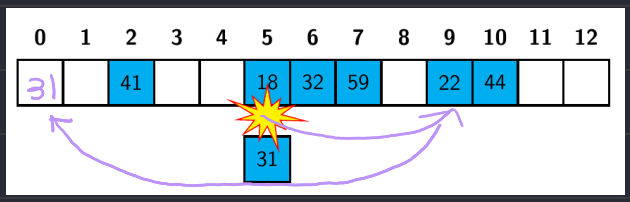
Definition

定義第二個 hash function, 通常是 $h'(k) = q - (k \mod q)$ 且 q < N, q 是質數

發生第 i 次碰撞時就計算新的 hash code = $h(k) + i \cdot h'(k)$

注意 h'(k) 不可以為0，否則會無法正確的 probing

Insertion



i=0 → $h(k) = h(31) = 5$

i=1 → $h(k) = h(31) + 1 \cdot h'(31) = 9$

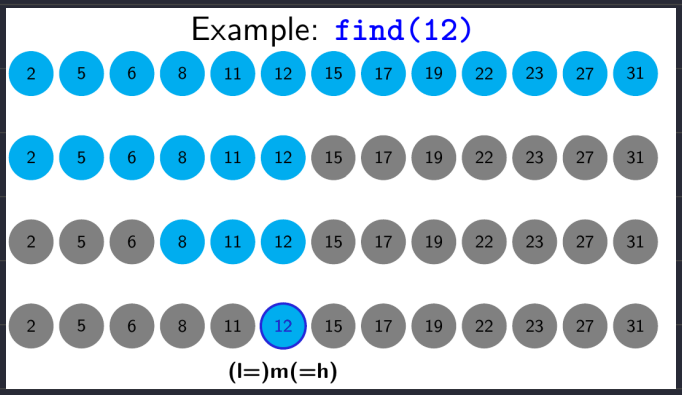
i=2 → $h(k) = h(31) + 2 \cdot h'(31) = 0$

Ordered Search Tables

Definition

A dictionary implemented by means of sorted array (lists)

Binary Search



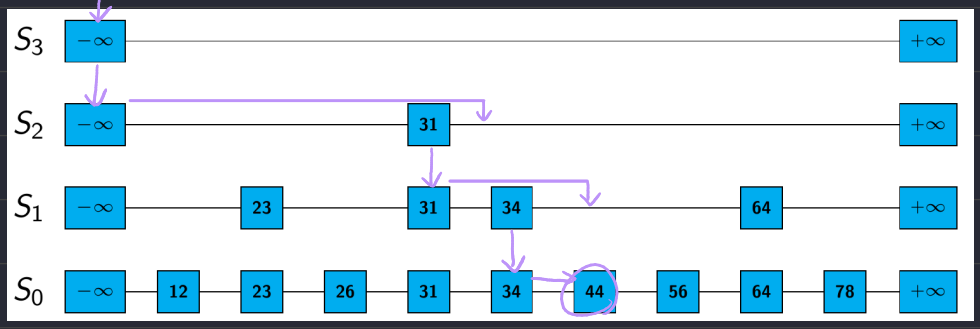
Comparing Different Implementations

Method	List	Hash Table	Search Table
`size`, `isEmpty`	$O(1)$	$O(1)$	$O(1)$
`entries`	$O(n)$	$O(n)$	$O(n)$
`find`	$O(n)$	$O(1)$ exp., $O(n)$ worst-case	$O(\log n)$
`findAll`	$O(n)$	$O(1+s)$ exp., $O(n)$ worst-case	$O(\log n + s)$
`insert`	$O(1)$	$O(1)$	$O(n)$
`remove`	$O(n)$	$O(1)$ exp., $O(n)$ worst-case	$O(n)$

Skip Lists

Search

e.g. Search for 44



Insertion

1. repeatedly toss a coin until we get tails,

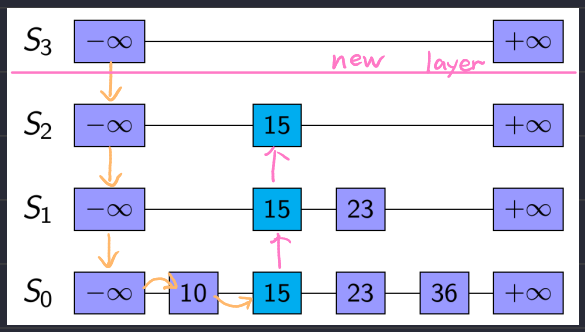
   and denote the number of times the coin came up heads with i

2. if i ≥ h, we add to the skip list new lists $S_{h+1}, \ldots, S_{i+1}$, each

   containing only the two special keys $-\infty, \infty$

3. 使用搜尋的方式找到正確位置並插入（最底層）

4. 開始往上把本次插入的值放到上方所有 layer 中，幫新增的 layer



insert 15,

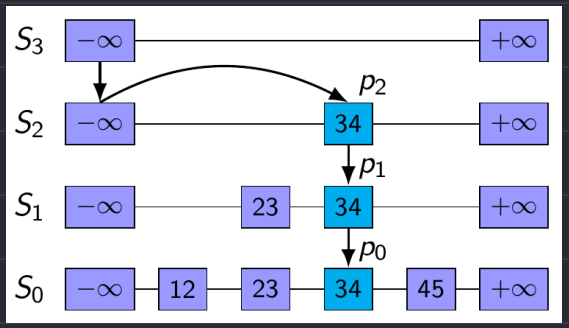
with i = 2 after tossing a coin

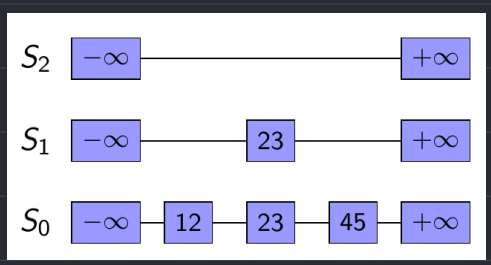
Removal

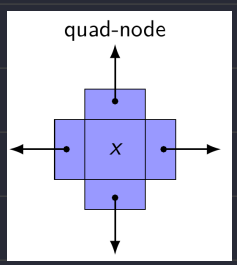
1. 往下找，直到遇到欲刪除的節點

2. 從找到的節點一路向下尋訪到最深的 layer 並 刪除所有尋訪的節點

3. 檢查最行方是否只有一個 layer 僅有 $-\infty, \infty$







Height of A Skip List

Fact Used

1. The probability of getting i consecutive heads when flipping a coin is $\frac{1}{2^i}$
2. If each of n events has probability p, the probability of event occurs np
3. If each of n entries is presented in a set with probability p → np
4. The expected number of coin tosses required in order to get tails is 2

Prvoe

1. By Fact 1, we insert an entry in list S_i with probability $\frac{1}{2^i}$
2. By Fact 3, $P_i = \frac{n}{2^i}$
3. The probability of height h ≥ i, E(h≥i) , is equal to Pi
   → h > i 代表第 i 層至少有一個元素
   → 如果我們把高度設為 clogn（其中 c 是一個常數），發生的機率是多少？

$$P_{c\log n} = \frac{n}{2^{c\log n}} = \frac{n}{(2^{\log n})^c} = \frac{n}{n^c} = \frac{1}{n^{c-1}}$$

4. Suppose $h \geq c\log n$, then $E_{(h \geq c\log n)} \leq \frac{1}{n^{c-1}}$
   → 也就是說不用插入新陣列的機率是 $E_{(h < c\log n)} \geq 1 - \frac{1}{n^{(c-1)}}$

→ Skip list 操作有很高機率是 O(log n)

Search and Update Times

O(log n)

Space Usage

O(n)